

CashUp

Análise de Negócio & Regras de Negócio

Sistema de Gestão Financeira Pessoal — iOS / SwiftUI

Documento gerado em 05/03/2026

Este documento apresenta a análise completa do domínio de negócio do aplicativo **CashUp**, mapeando todas as entidades, fluxos e regras que governam o comportamento do sistema. Também identifica lacunas funcionais e oportunidades de melhoria levantadas pela inspeção do código-fonte.

Sumário

1. Visão Geral do Sistema
 2. Domínio — Entidades Principais
 3. Regras de Negócio — Transações
 4. Regras de Negócio — Categorias
 5. Regras de Negócio — Planejamento Orçamentário
 6. Regras de Negócio — Dashboard e Métricas
 7. Regras de Negócio — Navegação Temporal
 8. Regras de Negócio — Segurança e Onboarding
 9. Lacunas Identificadas (Gap Analysis)
-

1. Visão Geral do Sistema

O **CashUp** é um aplicativo iOS de **gestão financeira pessoal** que permite ao usuário registrar receitas e despesas, organizá-las em categorias hierárquicas e planejar um orçamento mensal. Toda a persistência é **local**, utilizando **SwiftData** (sem sincronização remota). A moeda padrão é o **Real Brasileiro (BRL)**, com formatação pt_BR.

Arquitetura: MVVM com SwiftUI + SwiftData. ViewModels são classes ObservableObject anotadas com @MainActor. Combine é utilizado para sincronização reativa entre ViewModels.

Módulos funcionais identificados:

Módulo	Descrição
Transações	Registro de receitas e despesas, com suporte a recorrência
Categorias	Hierarquia Categoria → Subcategoria com seed de dados iniciais
Planejamento	Orçamento mensal por categoria/subcategoria
Dashboard (Home)	Resumo financeiro do mês com gráfico diário
Dicas	Previsto no código; não implementado
Autenticação	Biometria prevista; não implementada
Onboarding	Fluxo de boas-vindas previsto; não implementado

2. Domínio — Entidades Principais

Entidade	Responsabilidade	Atributos Principais
ExpenseModel	Transação (despesa ou receita)	amount, date, isIncome, repetition, categoria, subcategoria
CategoriaModel	Agrupamento temático	nome, icon, cor (RGB), subcategorias[]
SubcategoriaModel	Classificação específica	nome, icon, usageCount, categoria
CategoriaPlanejadaModel	Orçamento de categoria por mês	mesAno, categoriaOriginal, subcategoriasPlanejadas[]
SubcategoriaPlanejadaModel	Valor orçado por subcategoria	valorPlanejado, subcategoriaOriginal, categoriaPlanejada
RepetitionData	Dados de recorrência de transação	repeatOption, endDate?, excludedDates[]?

Relacionamentos:

- ExpenseModel → CategoriaModel (opcional, N:1)
- ExpenseModel → SubcategoriaModel (opcional, N:1)
- CategoriaModel → SubcategoriaModel[] (1:N, bidirecional via @Relationship inverse)
- CategoriaPlanejadaModel → SubcategoriaPlanejadaModel[] (1:N, cascade delete)
- CategoriaPlanejadaModel → CategoriaModel (referência à categoria original)

- SubcategoriaPlanejadaModel → SubcategoriaModel (referência à subcategoria original)

3. Regras de Negócio — Transações

RN-01 — Validação de Valor

- O valor (**amount**) de uma transação deve ser **estritamente maior que zero**.
- Valores zerados ou negativos são rejeitados antes do salvamento (*AddTransactionViewModel.criarTransacaoEChamarClosure*).
- Valores são armazenados como **Double** e exibidos com 2 casas decimais no formato BRL (R\$ 0,00).

RN-02 — Classificação: Despesa vs. Receita

- Toda transação é classificada como **Despesa** ou **Receita** via o flag *isIncome: Bool*.
- Transações vinculadas à categoria "**Renda**" são automaticamente marcadas como receita (*isIncome = true*), independentemente do tipo selecionado pelo usuário na UI.
- Ao criar uma **despesa**: a categoria "Renda" **não aparece** nas opções de seleção.
- Ao criar uma **receita**: **somente** a categoria "Renda" é apresentada.

RN-03 — Categorização Obrigatória

- Toda transação **deve ter categoria e subcategoria** associadas para ser salva.
- A ausência de qualquer um desses campos bloqueia o salvamento.
- A descrição (*expenseDescription*) é **opcional**.

RN-04 — Recorrência de Transações

Uma transação pode ser configurada como recorrente com as seguintes frequências:

Opção	Comportamento
Nunca	Padrão — transação pontual
Diariamente	Repete a cada 1 dia
Semanalmente	Repete a cada 7 dias
A cada 10 dias	Repete a cada 10 dias
Mensalmente	Repete a cada 1 mês
Anualmente	Repete a cada 1 ano

- Transações recorrentes **não geram registros físicos** para cada ocorrência — o sistema projeta instâncias virtuais (*DisplayableExpense*) calculando as datas dentro do intervalo mensal em tempo de exibição.
- A data de término da recorrência é **opcional**. Sem ela, o padrão sugerido na UI é **1 ano a partir da data inicial**.

RN-05 — Exclusão de Ocorrências Recorrentes

Ao excluir uma ocorrência de transação recorrente, o usuário seleciona o escopo:

Escopo	Comportamento no Banco
Esta ocorrência apenas	Adiciona a data às <code>excludedDates</code> da série; as demais ocorrências permanecem
Esta e todas as futuras	Define <code>endDate</code> da série para o dia anterior à ocorrência selecionada
Toda a série	Deleta o <code>ExpenseModel</code> original do banco de dados

- Se ao aplicar "Esta e futuras" a nova `endDate` resultante for **anterior à data de início** da série, toda a série é deletada como fallback.

4. Regras de Negócio — Categorias

RN-06 — Seed de Dados Iniciais

- O sistema popula **7 categorias pré-definidas** na **primeira inicialização** (quando não há categorias no banco).
- As categorias seed possuem **IDs fixos e determinísticos** (*SeedIDs*), usados em regras hardcoded do sistema.
- O seed nunca é re-executado se já existirem categorias — operação idempotente.

Categoria	Cor Padrão	Papel
Renda	Verde	Exclusiva para receitas; papel especial no sistema
Comidas e Bebidas	Teal	Despesa
Entretenimento	Rosa	Despesa
Habitação	Laranja	Despesa
Transporte	Roxo	Despesa
Estilo de Vida	Azul	Despesa
Diversos	Marrom	Despesa

RN-07 — Subcategorias Favoritas

- Cada subcategoria rastreia um **contador de uso** (*usageCount*), incrementado a cada seleção ao criar uma transação.
- As **6 subcategorias mais utilizadas** são exibidas como atalhos na tela de seleção de categoria.
- A lista de favoritos é filtrada pelo tipo de transação: subcategorias de "Renda" nunca aparecem em favoritos de despesas e vice-versa.

RN-08 — Edição de Categorias

- A edição de categorias está estruturada na UI (*CategoriesViewEdit*), mas as ações de **adicionar e deletar categorias/subcategorias** estão **comentadas** e não funcionais no estado atual.
- Somente visualização está ativa.

5. Regras de Negócio — Planejamento Orçamentário

RN-09 — Escopo Mensal

- O planejamento é **por mês/ano**. O campo *mesAno* sempre armazena o **primeiro dia do mês** (*startOfMonth()*).
- Uma mesma categoria **só pode ter um planejamento por mês**. Tentativas de adicionar uma categoria já planejada redirecionam para inclusão de subcategoria na entrada existente.

RN-10 — Estrutura Hierárquica

- O planejamento tem dois níveis: **Categoria** → **Subcategoria**.
- Uma subcategoria só pode aparecer **uma vez** no planejamento de uma categoria em um dado mês (unicidade verificada antes do insert).
- Cada subcategoria planejada possui um **valorPlanejado** ≥ 0 .
- O valor total planejado de uma categoria = Σ **valorPlanejado** das suas subcategorias.

RN-11 — Cascade Delete Automático

- Ao remover subcategorias planejadas, se a **categoria planejada ficar sem subcategorias**, ela é **automaticamente deletada**.
- Este comportamento é garantido tanto pela regra de negócio no ViewModel quanto pela configuração *deleteRule: .cascade* no modelo.

RN-12 — Cópia de Planejamento entre Meses

- É possível copiar o planejamento do mês atual para o **próximo mês**.
- Categorias que **já existem no mês de destino são ignoradas** (sem sobrescrever dados existentes).
- Os valores planejados das subcategorias são **copiados integralmente**.
- Se não há planejamento no mês atual, a operação é bloqueada com alerta ao usuário.

RN-13 — Zerar Planejamento do Mês

- O usuário pode **zerar completamente** o planejamento do mês atual.
- Esta ação deleta todos os registros *CategoriaPlanejadaModel* do mês (e por cascade todas as *SubcategoriaPlanejadaModel* associadas).

6. Regras de Negócio — Dashboard e Métricas

RN-14 — Cálculos Financeiros do Mês

Métrica	Fórmula
Total Gasto	Σ amount de todas as despesas do mês
Total de Receitas	Σ amount de todas as receitas do mês
Total Planejado	Σ valorPlanejado de todas as subcategorias planejadas do mês
Saldo Restante	Total Planejado – Gasto em Categorias Planejadas
Progresso por Categoria	$\min(\text{gasto} / \text{planejado}, 1.0)$ — limitado a 100%

- O **Saldo Restante** considera apenas gastos em subcategorias que estão **dentro do planejamento**. Gastos em subcategorias não planejadas **não reduzem** o saldo restante.
- As categorias no dashboard são exibidas em **ordem decrescente** de gasto total.

RN-15 — Gráfico Diário de Despesas

- O gráfico exibe um ponto para **cada dia do mês**, mesmo que o valor seja zero.
- Transações recorrentes contribuem para o gasto do dia da **ocorrência virtual**, não apenas para a data original da série.

7. Regras de Negócio — Navegação Temporal

RN-16 — Mês Sincronizado

- O mês navegado é **sincronizado entre Home, Planejamento e Despesas** via publishers Combine (*objectWillChange* + *@Published* sinks).
- A data de referência interna é sempre o **início do mês** (*startOfMonth()* — dia 1, 00:00:00).
- A navegação é incremental: **±1 mês** por vez, usando *Calendar.date(byAdding: .month)*.
- Ao mudar o mês em qualquer tela, as outras são notificadas e recalculam seus dados automaticamente.

8. Regras de Negócio — Segurança e Onboarding

RN-17 — Autenticação Biométrica (Planejada)

- A estrutura para **Face ID / Touch ID** está prevista (*AuthViewModel*, *BiometricView*).
- Os ViewModels correspondentes estão **vazios** — funcionalidade **não implementada**.

RN-18 — Onboarding e Dicas (Planejados)

- Existe estrutura para fluxo de boas-vindas (*OnboardingView*, *WelcomeView*) e seção de dicas financeiras (*TipsView*).
- Os ViewModels correspondentes (*OnboardingViewModel*, *TipsViewModel*, *Tip*) estão **vazios**.

9. Lacunas Identificadas — Gap Analysis

As lacunas abaixo foram identificadas pela inspeção do código-fonte. Representam funcionalidades ausentes, comentadas ou estruturadas mas não implementadas.

ID	Impacto	Descrição
L-01	Alto	Criação/edição/exclusão de categorias customizadas desabilitada — código comentado em CategoriesViewEdit
L-02	Alto	Autenticação biométrica não implementada — dados financeiros pessoais sem proteção de acesso
L-03	Médio	Validação de valor negativo apenas no ViewModel, não no Model — integridade fraca no nível de dados
L-04	Médio	Sem suporte a múltiplas moedas — locale fixo em pt_BR/BRL

L-05	Médio	Sem exportação de dados (PDF/CSV) — sem portabilidade para o usuário
L-06	Médio	Sem metas de poupança — funcionalidade chave de gestão financeira ausente
L-07	Médio	Sem alertas de orçamento excedido (push notifications) — usuário não é notificado proativamente
L-08	Baixo	Dicas financeiras (TipsView/TipsViewModel) não implementadas
L-09	Baixo	usageCount não é decrementado ao excluir transação — favoritos podem ficar desatualizados
L-10	Baixo	Sem sincronização iCloud — dados perdidos ao trocar de dispositivo

Documento gerado automaticamente por análise de código-fonte — CashUp v0.1 · 2026